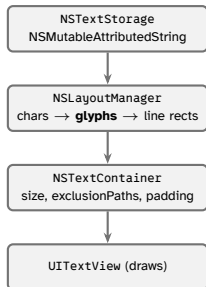


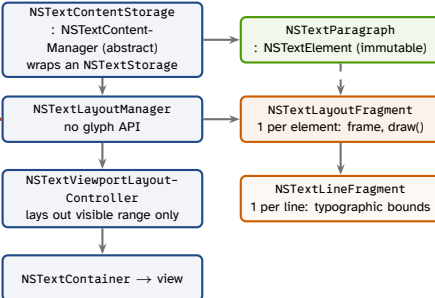
In one line: TextKit is the engine under UITextView and UITextField: a **storage** (the attributed text) → a **layout manager** → a **text container** (the shape to fill) → the **view**. **TextKit 2** (iOS 15, default for UITextView since **iOS 16**) swaps glyph-based, whole-document layout for **element/fragment** objects laid out **only around the viewport**. Touching the old .layoutManager silently and **permanently** drops a text view back to TextKit 1.

TextKit 1 (iOS 7)



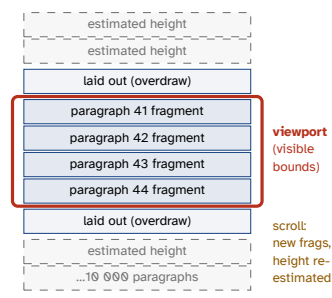
NSRange (UTF-16) everywhere;
non-contiguous layout optional

TextKit 2 (iOS 15+)



NSTextLocation / NSTextRange (objects, not ints);
layout always non-contiguous; renders via Core Text

Viewport layout



delegate: WillLayout → configureRendering-
SurfaceFor: (per fragment) → DidLayout

How it works

- TK1:** NSLayoutManager maps chars to **glyphs** and lays out *from the document start* to whatever you query. Glyph APIs assume chars ↔ glyphs are contiguous — false for Indic/Arabic scripts; TK2 has none.
- TK2:** the content manager splits text into immutable **elements**; the layout manager makes one **layout fragment** per element (layoutFragmentFrame, textLineFragments, draw(at:in:)); the viewport controller decides which exist. Cost scales with the **screen**, not the document.
- Pick per view: UITextView(usingTextLayoutManager: false) (iOS 16) or IB “Text Layout”. TK1 view ⇒ textLayoutManager is nil.
- Compatibility mode:** reading textView.layoutManager (or textContainer.layoutManager) rebuilds the view on TK1 — **expensive, loses UI state, no way back**. Catch it: breakpoint _UITextViewEnablingCompatibilityMode.
- Edits:** mutate the backing NSTextStorage inside performEditingTransaction {} so elements and fragments refresh.
- Customise, storage untouched:** textContentStorage(:textParagraphWith:) (display-only attributes, e.g. dim comments) · textLayoutManager(:textLayoutFragmentFor:in:) returns your fragment subclass (bubbles, code-block backgrounds). NSTextList in UIKit since iOS 16.

Example — count lines the TK2 way (no glyphs)

```

let tv = UITextView(usingTextLayoutManager: true)
guard let tlm = tv.textLayoutManager else { return } // nil = TK1
var lines = 0
tlm.enumerateTextLayoutFragments(from: tlm.documentRange.location,
options: [.ensuresLayout]) { frag in // forces layout: costly
    lines += frag.textLineFragments.count
    return true // false = stop early
}
let cs = tlm.textContentManager as? NSTextContentStorage
cs?.performEditingTransaction { // edit so TK2 sees it
    cs?.textStorage?.append(NSAttributedString(string: "\nmore"))
}
// DON'T: tv.layoutManager.numberOfGlyphs -> falls back to TK1
  
```

Which view?

UILabel	display only: no selection, no tappable links, no scroll; cheapest
UITextField	one line of input; TK2 since iOS 15
UITextView	selectable/editable, links, attachments; a scroll view — isScrollEnabled = false to self-size
SwiftUI	Text to display · TextEditor to edit (rich in iOS 26)
Core Text	you draw: custom layout, columns, text on paths

Attachments & wrapping

NSTextAttachment = one U+FFFC character carrying an image; TK2 can host a live view (NSTextAttachmentViewProvider, iOS 15). Wrap around a shape: textContainer.exclusionPaths (container coordinates).

Core Text — when TextKit is too high-level

- C API under TextKit: CTFramesetter → CTFrame (fills a CGPath) → CTLines → CTRuns (one attribute set each). Size first: CTFramesetterSuggestFrameSizeWithConstraints.
- Bottom-left origin:** flip the CGContext (scaleBy(x: 1, y: -1)) and reset textMatrix. No selection, editing, a11y or hit-testing for free.

Big documents — performance

- Height outside the viewport is **estimated**: usageBoundsForTextContainer and the scroll indicator shift while scrolling. .ensuresLayout over documentRange lays out *everything* — keep it off the hot path.
- Batch: beginEditing()/endEditing() (or one transaction) = one layout pass.
- Syntax highlighting: restyle only the *edited paragraph* (NSTextStorageDelegate didProcessEditing), not the whole text per key.
- Measure without a view: boundingRect(with:options:context:) + .usesLineFragmentOrigin — then ceil.

Traps

- One stray .layoutManager** — yours or a library's — silently turns that text view into TK1. Grep for it.
- “No glyphs, so no line rect” — enumerate fragments / textLineFragments.
- Mutating textStorage outside performEditingTransaction → stale layout.
- UITextView vs UILabel misaligned: textContainerInset (8,0,8,0) + lineFragmentPadding 5 — zero both.
- NSRange counts **UTF-16** units, not Characters: NSRange(range, in: string).

Remember

“Storage → elements → fragments → viewport. Touch the layout manager and you’re back in 2013.”